

Below is a **master table of major software principles used in the IT industry, and when to use which principle** (practically, in real projects).

✓ **MASTER TABLE — Software Principles (IT Industry) + When to Use What**

#	Principle / Rule	Meaning (Short)	When to Use / Best Use Case	Common Examples
1	KISS (Keep It Simple, Stupid)	Keep solutions simple	When design becomes too complex / over-engineered	Microservices vs Monolith decision, clean APIs
2	DRY (Don't Repeat Yourself)	Avoid duplication	When same logic appears in multiple places	Common validation, shared utility methods
3	WET (Write Everything Twice)	Duplicate until required	When abstraction is premature	Early-stage products, MVP builds
4	YAGNI (You Aren't Gonna Need It)	Don't build future features now	When features are speculative	Avoid building unused modules
5	SOLID	Object-Oriented design principles	When designing scalable OOP systems	Java, C#, Python OOP architecture
6	SRP (Single Responsibility)	One class = one job	When classes become large or overloaded	Separate Service / Repository / Controller
7	OCP (Open Closed)	Extend without modifying	When adding new behaviors often	Strategy pattern, plugins
8	LSP (Liskov Substitution)	Subclasses must behave like base	When using inheritance heavily	Avoid breaking polymorphism
9	ISP (Interface Segregation)	Small focused interfaces	When interfaces get too big	Separate payment interface vs full account interface
10	DIP (Dependency Inversion)	Depend on abstractions	When system must be testable and replaceable	DI containers, Spring Boot

11	GRASP principles	Responsibility assignment patterns	When designing object responsibilities	Information Expert, Controller pattern
12	Separation of Concerns (SoC)	Split system responsibilities	Always (core architecture principle)	UI separate from business logic
13	Encapsulation	Hide internal state	When exposing data can break logic	Private fields + getters/setters
14	Abstraction	Show necessary details only	When designing APIs & layers	Interface-based coding
15	Modularity	Build independent modules	For maintainability and scalability	Maven modules, Python packages
16	High Cohesion	Each module focused	When code feels messy	Clean service boundaries
17	Low Coupling	Reduce dependency	Always for maintainable code	Avoid tight integration between modules
18	Law of Demeter (LoD)	Talk only to immediate friends	When chained method calls appear	a.getB().getC().do() is warning
19	Principle of Least Astonishment (POLA)	Behave as expected	When designing UI/UX/API behavior	Simple predictable naming/flow
20	Principle of Least Privilege	Minimum access rights	Always (security principle)	IAM roles, DB permissions
21	Fail Fast	Detect errors early	For validation and correctness	Input validation, asserts
22	Defensive Programming	Handle bad input safely	For risky systems / user inputs	Null checks, boundary validations
23	Robustness Principle (Postel's Law)	Liberal in input, strict in output	APIs integration handling	Accept variations, return consistent schema
24	Design by Contract (DbC)	Pre/postconditions	In safety-critical systems	Contracts, invariants

25	Single Source of Truth (SSOT)	One authoritative data source	When data duplication causes conflicts	Central config store, master DB
26	Convention Over Configuration	Reduce manual configs	When building frameworks / standard org apps	Spring Boot defaults
27	Immutability Principle	Avoid mutable state	When concurrency or bugs exist	final, immutable DTOs
28	Idempotency	Same request = same effect	In distributed systems	Payment retries, REST PUT
29	Consistency Principle	Same patterns everywhere	For large teams	Naming conventions, API style guides
30	Reusability Principle	Build components reusable	When multiple projects share modules	Shared libraries, SDKs
31	Extensibility Principle	Add future features easily	When roadmap changes often	Plugin architectures
32	Maintainability Principle	Optimize for readability & edits	Always	Simple code over clever code
33	Testability Principle	Code should be testable	Always in modern IT	DI + unit tests
34	Observability Principle	Track system behavior	In production systems	Logs + metrics + tracing
35	12-Factor App	Cloud-native best practices	Cloud apps/microservices	Stateless services, env configs
36	CAP Theorem	Choose C/A/P tradeoff	Distributed DB/system design	Cassandra (AP), Postgres (CP-like)
37	ACID	Reliable transaction guarantee	Banking/finance/critical systems	RDBMS transactions
38	BASE	Eventually consistent systems	High-scale distributed systems	DynamoDB, Cassandra
39	Eventual Consistency	Updates sync over time	Distributed systems	Async systems, replication

40	Unix Philosophy	Do one thing well	Tooling & DevOps design	CLI utilities, microservices design
41	Simplicity over Cleverness	Prefer readable code	Always for teams	Reduce one-liners/hacks
42	Avoid Premature Optimization	Optimize only when needed	Performance issues after profiling	Profile first, then optimize
43	Performance Principle	Optimize bottlenecks	When SLAs fail	Caching, indexing
44	Scalability Principle	Handle growth	When traffic expected to grow	Load balancing, sharding
45	Reliability Principle	System must work consistently	For production enterprise apps	Retry, circuit breaker
46	Availability Principle	Minimize downtime	Mission-critical systems	HA, failover
47	Security by Design	Secure from start	Any system	Threat modeling, secure SDLC
48	Privacy by Design	Protect personal data	GDPR-like systems	Data minimization
49	Zero Trust Principle	Never trust, always verify	Modern enterprise security	MFA, device-based auth
50	Keep Backward Compatibility	Do not break consumers	APIs and public services	API versioning
51	Loose Coupling via Contracts	Integrate via contract	Microservices	OpenAPI, gRPC protobufs
52	Don't Break Production	Release discipline	Any org with live users	Feature flags, canary deployments
53	Documentation Principle	Document essential knowledge	When teams scale	README, ADR, API docs
54	Refactoring Principle	Improve without changing behavior	When tech debt accumulates	Clean up codebase regularly

55	Technical Debt Principle	Debt must be managed	All long-term projects	Tech debt backlog
56	Lean Principle	Reduce waste, optimize value	Fast-moving product teams	MVP, short cycles
57	Agile Principles	Customer + iterative delivery	For evolving requirements	Scrum/Kanban
58	DevOps Principles	Collaboration + automation	Continuous delivery systems	CI/CD, infra automation
59	SRE Principles	Reliability engineering	High-availability systems	SLO, error budgets
60	Data Governance Principles	Manage data as an asset	Enterprises	Data lineage, access control

□ Quick “Which Principle to Use When” Map

Situation	Best Principles
Codebase becomes too complex	KISS, SRP, SoC, High cohesion
Same logic repeated everywhere	DRY, Reusability
Team is over-engineering early	YAGNI, WET
Want scalable architecture	SOLID, Modularity, Low coupling, Extensibility
Designing microservices	12-Factor, CAP, Idempotency, Observability
Building secure system	Least privilege, Zero Trust, Security by design
DB transactions critical	ACID, SSOT
Distributed system at scale	BASE, Eventual consistency, CAP
Too many production issues	SRE principles, Reliability, Observability
Release failures	Don't break production, Feature flags, Backward compatibility

If you want, I can also provide:

- ✓ Printable one-page cheat sheet
- ✓ Principle → Anti-Pattern → Fix mapping table (like 50+ rows)
- ✓ Principle-by-principle Java / Python mini code examples

Absolutely ✓

Below is the **Principle → Anti-Pattern → Fix mapping MASTER table** with **50+ rows** (industry-grade).

✓ Principle → Anti-Pattern → Fix Mapping Table (MASTER — 60 Rows)

#	Principle	Anti-Pattern (Wrong Practice)	Fix / Correct Approach
1	KISS	Over-engineering simple features	Build simplest working solution first
2	KISS	Too many layers for small app	Reduce layers, keep minimum required abstractions
3	DRY	Copy-paste code blocks everywhere	Extract common method/service/module
4	DRY	Duplicate validation logic	Use shared validator library/middleware
5	YAGNI	Building features “for future”	Build only what is required now
6	YAGNI	Adding unused microservices	Start monolith, split only when needed
7	WET	Premature abstraction	Duplicate twice, abstract after patterns emerge
8	SOLID	“God class” (one class does all)	Apply SRP, split responsibilities
9	SRP	Service handles DB + logic + UI	Separate Controller/Service/Repository
10	SRP	One method 500 lines long	Break into small functions with 1 task
11	OCP	Editing old code for every new rule	Use Strategy/Factory/Plugin architecture
12	OCP	Big switch-case for behavior	Replace with polymorphism or strategy map

13	LSP	Subclass changes meaning of parent method	Redesign inheritance; prefer composition
14	LSP	Child throws unsupported exception	Use interfaces or split hierarchy
15	ISP	Huge interface with 30 methods	Split into small role-based interfaces
16	ISP	Clients forced to implement unused methods	Create multiple specialized interfaces
17	DIP	Direct dependency on concrete classes	Depend on interfaces/abstractions
18	DIP	Hard-coded dependencies	Use Dependency Injection (DI container)
19	Low Coupling	One module depends on many others	Introduce interfaces + events/message bus
20	Low Coupling	Tight integration with DB schema	Use repository layer + migrations
21	High Cohesion	Module contains unrelated features	Split into meaningful bounded modules
22	High Cohesion	"Utils" class with random methods	Create domain-specific utilities/services
23	SoC	UI logic mixed with business logic	Keep presentation separate from logic
24	SoC	DB query inside controller	Move DB logic to repository layer
25	Encapsulation	Public fields modified anywhere	Use private fields + controlled access
26	Encapsulation	External code changes object state directly	Provide methods enforcing business rules
27	Abstraction	Leaking implementation details in API	Expose interface/DTO instead of internal model
28	Abstraction	Over-abstracting everything	Keep only useful abstractions
29	Law of Demeter	Long chained calls: a.b().c().d()	Add methods or simplify object interactions
30	Least Privilege	Giving admin permissions to service	Create limited roles/scopes

31	Security by Design	Security added at end	Threat modeling + secure SDLC early
32	Security by Design	Secrets in code or Git	Use Vault/Secrets Manager + env vars
33	Fail Fast	Errors detected late in pipeline	Validate inputs early, stop early
34	Fail Fast	Ignoring errors and retry blindly	Detect + log + exit early with clear reason
35	Defensive Programming	Trusting user input	Validate, sanitize, normalize inputs
36	Postel's Law	Strict input rejection everywhere	Accept flexible input formats, normalize
37	Idempotency	Retry causes double payment/order	Use idempotency keys + safe retry logic
38	Idempotency	POST used for updates always	Use PUT/PATCH for idempotent updates
39	Consistency	Different naming/style per developer	Enforce linting, style guide, templates
40	Backward Compatibility	Breaking API changes without notice	Version APIs + deprecation policy
41	SSOT	Multiple sources hold same truth	Centralize master data + sync strategy
42	Maintainability	Clever code that no one understands	Prefer readability over smart hacks
43	Maintainability	No refactoring ever	Continuous refactoring + tech debt backlog
44	Refactoring Principle	"If it works don't touch it"	Improve code quality without changing behavior
45	Testability	Static singletons everywhere	DI + mockable interfaces
46	Testability	Tight coupling to DB in unit tests	Use repository mocks + integration tests separately
47	Observability	No logs, no metrics	Add logs + metrics + traces
48	Observability	Logs without correlation IDs	Add traceId/requestId propagation

49	Performance	Optimizing without measurement	Profile first, optimize bottlenecks only
50	Premature Optimization	Micro-optimizing early stage code	Improve readability, optimize later
51	Scalability	Single-node design for high traffic	Horizontal scaling + cache + DB sharding
52	Reliability	No retry/backoff strategy	Use exponential backoff + circuit breaker
53	Reliability	Retry storm under failure	Circuit breaker + bulkheads + rate limiting
54	Availability	Single point of failure	HA setup + redundancy + failover
55	12-Factor App	Storing config in code	Store configuration in environment
56	12-Factor App	Stateful services	Externalize state (DB, Redis, S3)
57	ACID	Partial update without transaction	Use DB transactions
58	BASE / Eventual Consistency	Expecting instant consistency in distributed system	Design async workflows + reconciliation
59	DevOps Principle	Manual deployment steps	Use CI/CD pipelines
60	SRE Principle	"99.9 uptime" with no SLO/SLI	Define SLOs, error budgets, alert policy

★ Extra (Bonus) High-Impact Anti-Pattern → Correct Pattern Rows

#	Principle	Anti-Pattern	Fix
61	Modularity	Monolithic unstructured codebase	Split into modules with clean boundaries
62	Reusability	Copying code between services	Build shared libraries/SDK
63	Documentation	No README / no API docs	Maintain README + OpenAPI/Swagger docs
64	Clean Architecture	Business logic depends on frameworks	Frameworks depend on business core

65	Agile	"Fake Agile" = no documentation, chaos	Agile with discipline: backlog + sprint planning + retros
66	Lean	Building massive features at once	MVP + small iterations
67	Concurrency Safety	Shared mutable state	Use immutability, locks, message queues
68	Privacy by Design	Collecting too much user data	Data minimization + masking + retention policy
69	Zero Trust	Trusting internal traffic	mTLS, authorization, continuous validation
70	Resilience	Assuming dependencies always work	Timeouts + fallback + graceful degradation

✔ If you want next:

I can now generate **one massive printable PDF cheat-sheet** and also provide:

- ✔ **Principle → Anti-pattern → Fix → Real software example (Netflix/Google/AWS/Kubernetes)**
- ✔ **Microservices-only version**
- ✔ **Java + Spring Boot mapping examples (code)**

Below is the upgraded **industry-grade mapping**:

✔ **Principle → Anti-Pattern → Fix → Real Software Example**

(Netflix / Google / AWS / Kubernetes style examples)

Note: I'm using widely known, commonly documented architectural practices/patterns from these ecosystems (not "secret internal code").

✓ MASTER TABLE (50 Rows)

#	Principle	Anti-Pattern	Fix / Best Practice	Real Software Example
1	KISS	Too many microservices for small app	Start simple (monolith or modular monolith)	Many AWS startup guides recommend small → scale later
2	KISS	Complex custom orchestration	Use managed tools	AWS EKS / ECS instead of custom infra
3	DRY	Copy-paste same logic across services	Shared library / shared API	Google internal libraries (shared services/utilities concept)
4	DRY	Duplicate auth logic everywhere	Central auth + middleware	Kubernetes uses centralized RBAC policies
5	YAGNI	Building features not requested	Build minimal working system	Lean MVP approach (common in cloud-native startups)
6	YAGNI	Pre-splitting into microservices	Split when scaling demands it	Netflix began with monolith then evolved
7	SRP	One service does everything	Split responsibilities cleanly	Kubernetes separates Scheduler / Controller / API server
8	SRP	A class handles DB + business + API	Layered architecture	Typical Spring Boot layered (Controller-Service-Repository)
9	SoC	Mix UI + business logic	Separate presentation/domain/data	Google's Material apps follow separation patterns
10	SoC	DB queries in controllers	Use repository/data layer	AWS reference architectures show layered separation
11	OCP	Modify old code for every new rule	Strategy + plugin architecture	Kubernetes admission controllers extend behavior

12	OCP	Large if-else for payment providers	Implement provider strategies	AWS Marketplace / payment routing style use-case
13	LSP	Subclass breaks base contract	Avoid wrong inheritance; use composition	Java libraries follow strict substitutability
14	ISP	One huge interface	Split into role-based APIs	Kubernetes API resources separated by responsibilities
15	DIP	Direct dependency on low-level code	Depend on abstractions	Spring Dependency Injection container
16	DIP	Hardcoded cloud provider code	Abstraction layer	Kubernetes Cloud Controller Manager concept
17	Encapsulation	Public mutable data everywhere	Private state + methods	Strong encapsulation in Google Java style guide practices
18	Abstraction	API leaks internal DB schema	DTO + API contracts	AWS APIs hide implementation (service contracts)
19	Law of Demeter	Deep chained calls	Keep interactions local	Microservices prefer API boundaries, not deep object graphs
20	Low Coupling	Services share DB	DB-per-service + events	Netflix microservices avoid shared DB approach
21	High Cohesion	Random features inside one service	Domain-based services	Google domain-driven systems & service ownership model
22	SSOT	Multiple “truth databases”	Single authoritative store	AWS Control Plane (single source for infra state)
23	Fail Fast	Errors discovered late	Validate early	Kubernetes API server validation rejects invalid configs

24	Defensive Programming	Trusting client input	Validate + sanitize	AWS API Gateway validation & request constraints
25	Idempotency	Retries create duplicate orders	Idempotency keys	AWS API best practice: idempotent retry tokens
26	Idempotency	Retry causes double execution	Exactly-once simulation	Kafka consumers + dedupe patterns (Netflix uses Kafka heavily)
27	Observability	No tracing/logging	Traces + logs + metrics	Google Cloud Operations (Stackdriver concept)
28	Observability	Logs without requestId	Correlation IDs	Kubernetes tracing/telemetry integrations
29	Security by Design	Security added at end	Secure SDLC + threat modeling	AWS Well-Architected Security Pillar
30	Least Privilege	Admin permissions to all services	Scoped IAM/RBAC	AWS IAM roles + Kubernetes RBAC
31	Zero Trust	Trust internal traffic blindly	mTLS + auth everywhere	Istio service mesh on Kubernetes enables mTLS
32	Privacy by Design	Collect all user data	Data minimization	Google privacy/security engineering approach
33	Backward Compatibility	Breaking API changes	Versioning + deprecation	Kubernetes API versioning (v1, v1beta1)
34	Consistency	Each service follows different API style	Standards + linters	Google API design guidelines
35	Maintainability	Complex codebase no refactoring	Continuous refactoring	Google engineering culture promotes code health
36	Avoid Premature Optimization	Optimize before profiling	Measure & profile first	Google SRE: measure SLI before optimizations
37	Performance Principle	Slow DB due to missing indexes	Indexes + caching	AWS RDS performance tuning

38	Scalability	Single node architecture	Horizontal scaling	Kubernetes scaling (ReplicaSet + HPA)
39	Availability	Single point of failure	Multi-zone redundancy	AWS Multi-AZ deployments
40	Reliability	No timeout / infinite waits	Timeouts everywhere	Netflix resilience engineering standards
41	Resilience	Retry storm under failure	Circuit breaker + backoff	Netflix Hystrix (classic circuit breaker)
42	Graceful Degradation	Entire app fails if one dependency fails	Fallback mechanisms	Netflix fallback content delivery style
43	Bulkhead Isolation	One overloaded service brings all down	Separate thread pools/limits	Netflix bulkhead patterns
44	Rate Limiting	Unlimited API usage	Quotas/throttling	AWS API Gateway throttling
45	12-Factor App	Config in code	Config in env/secret manager	Kubernetes ConfigMap & Secrets
46	12-Factor App	Stateful web apps	External state	Kubernetes stateless services + DB externalization
47	ACID	Partial DB writes	Transactions	Banking apps commonly rely on RDBMS ACID
48	BASE	Expecting strong consistency globally	Eventual consistency	AWS DynamoDB design philosophy
49	Automation (DevOps)	Manual deployments	CI/CD pipelines	AWS CodePipeline / GitHub Actions / ArgoCD
50	SRE Principle	Alert on everything	SLO-based alerting	Google SRE error budgets

★ Extra “Big-Tech level” mapping (Bonus 15 rows)

#	Principle	Anti-Pattern	Fix	Real Example
51	Immutable Infrastructure	SSH into prod servers	Replace instances via CI/CD	AWS Auto Scaling + AMI model
52	GitOps	Manual kubectl in prod	Git as single control plane	ArgoCD / Flux with Kubernetes
53	Declarative Config	Imperative manual changes	Infrastructure as Code	Kubernetes YAML manifests
54	Self-Healing	Manual restart	Liveness/Readiness probes	Kubernetes probes auto-restart unhealthy pods
55	Autoscaling	Manual scaling	HPA/VPA	Kubernetes HPA
56	Load Balancing	Single endpoint	Multi-instance + LB	AWS ALB / NLB
57	Caching	DB hit for everything	Cache hot reads	Netflix caching, AWS ElastiCache
58	CDN Principle	Serving large media from origin	CDN edge caching	Netflix + CDN concept, AWS CloudFront
59	Chaos Engineering	Assuming system won't fail	Test failures intentionally	Netflix Chaos Monkey
60	Data Partitioning	One large DB	Sharding/partitioning	High-scale systems like DynamoDB + partition strategy

If you want, I can now generate:

- ✓ **Microservices-only mapping (100 rows)**
- ✓ **Kubernetes-only mapping (80 rows)**
- ✓ **One printable PDF cheat sheet with this table**
- ✓ **Architecture diagram images for each example (Netflix/AWS/K8s style)**

Here is the **Microservices-only Principle → Anti-pattern → Fix mapping (MASTER = 100 Rows)**.

Focus: **microservices architecture, distributed systems, APIs, resilience, DevOps, observability, security, data consistency**

✓ Microservices-Only Mapping Table (100 Rows)

#	Principle / Rule	Anti-Pattern (Wrong)	Fix / Correct Pattern (Right)
1	Single Responsibility Service	One service handles many domains	Split by bounded context/domain ownership
2	Bounded Context (DDD)	Service boundaries based on tech layers	Define boundaries using business domains
3	High Cohesion	One service = random features	Group features by business capability
4	Low Coupling	Services tightly dependent	Contract-first integration + eventing
5	Independent Deployability	Services must be deployed together	Remove runtime dependencies, decouple releases
6	Team Ownership	Nobody owns service end-to-end	Assign service owner + SLO accountability
7	API First	DB-first exposure	Design API contract first (OpenAPI/Protobuf)
8	Strong Contracts	Informal undocumented APIs	Versioned schemas + API docs
9	API Versioning	Breaking changes in same version	/v1, /v2 or header-based versioning
10	Backward Compatibility	Consumers break after deployment	Add fields only, never remove abruptly
11	Consumer-Driven Contracts	Only provider tests APIs	Add CDC tests (Pact style)
12	Schema Evolution	Breaking schema updates	Forward/backward compatible change strategy
13	Separation of Concerns	Controller contains business logic	Clean layers: Controller → Service → Repo
14	KISS	Too many services early	Start modular monolith; split gradually

15	YAGNI	Services created “for scale”	Split only when needed by load/team growth
16	DRY vs Duplication Balance	Shared library causes coupling	Prefer duplication over shared runtime dependency
17	Loose Coupling via Events	Synchronous chaining of services	Use async events for non-critical flows
18	Asynchronous by Default	Everything uses REST synchronous	Use Kafka/RabbitMQ for workflows
19	Avoid Chatty Services	Many small calls per request	Use API composition/BFF/GraphQL
20	API Gateway Pattern	Clients call all services directly	Use API Gateway for routing, policies
21	BFF Pattern	One API for all clients	Separate BFF for mobile/web/internal
22	Service Mesh	Each service manages mTLS manually	Use Istio/Linkerd for uniform policy
23	No Shared Database	Multiple services write same DB	DB-per-service; integrate via events
24	Data Ownership	One service updates another service’s tables	Only owner service mutates its data
25	Database per Service	Shared schema coordination nightmare	Separate schemas/databases per service
26	CQRS	Same model for read/write	Separate read models for performance
27	Event Sourcing	State overwritten only	Store events, rebuild state if needed
28	SAGA Pattern	Distributed transaction (2PC)	Use Saga choreography/orchestration
29	Compensating Transactions	No rollback strategy	Design compensation for each step
30	Eventual Consistency	Expect strong consistency always	Use reconciliation + async processing

31	Idempotency	Retry creates duplicates	Idempotency keys + dedupe storage
32	Exactly-once Illusion	Assume broker delivers once	Design for at-least-once + idempotent consumers
33	Outbox Pattern	DB update without event publish	Transactional outbox + CDC
34	Inbox Pattern	Duplicate event processing	Store message IDs to prevent duplicates
35	Reliable Messaging	Fire-and-forget messages	Retry + DLQ + monitoring
36	Dead Letter Queue	Failed messages lost	Use DLQ with reprocessing strategy
37	Retry Policy	Infinite retries	Retry limit + backoff + jitter
38	Timeouts	No timeout leads to resource leak	Enforce timeouts for all network calls
39	Circuit Breaker	Retry storm causing outage	Circuit breaker + fallback
40	Bulkhead Isolation	One failure blocks everything	Separate pools/limits per dependency
41	Rate Limiting	Unlimited requests allowed	Rate limit per client/service
42	Backpressure	Overload cascades	Queueing + throttling + shedding load
43	Graceful Degradation	All-or-nothing system	Fallback responses, partial data return
44	Fail Fast	Requests hang forever	Fail quickly and return meaningful error
45	Resilience Testing	Assume infra never fails	Inject faults + chaos testing
46	Observability (3 pillars)	No logs/metrics/traces	Use logs + metrics + tracing everywhere
47	Correlation IDs	Can't trace request flow	Propagate traceId/requestId across services
48	Distributed Tracing	Only local logging	OpenTelemetry instrumentation

49	Structured Logging	Plain text logs	JSON logs with fields & IDs
50	Centralized Logging	Logs stored in servers	Log aggregation: ELK/Loki/CloudWatch
51	Metrics-first Alerts	Alert on logs only	Define metrics + thresholds
52	SLI/SLO	Alerts based on gut feeling	SLO-based alerting
53	Error Budgets	Release always prioritized	Use error budgets to control release velocity
54	Health Checks	No liveness/readiness	Add health endpoints + probes
55	Readiness vs Liveness	One health check used for all	Separate readiness and liveness
56	Auto Scaling	Manual scaling only	HPA/VPA + autoscaling policies
57	Immutable Deployments	Patch prod servers manually	Redeploy immutable images
58	Blue-Green Deploy	Deploy on same instances	Use blue-green for safe release
59	Canary Releases	100% rollout directly	Gradual rollout + monitoring
60	Feature Flags	Deploy equals release	Separate deployment & feature release
61	Rollbacks	No rollback plan	Version rollback + DB migration strategy
62	Database Migrations	Breaking migration in prod	Expand-Contract migrations
63	Compatibility Window	Old and new versions conflict	Maintain compatibility during rollout
64	Configuration Management	Config hardcoded	External config + secrets managers
65	Secret Management	Secrets in Git	Vault/KMS/Secrets Manager
66	Zero Trust	Trust internal network	mTLS + authN/authZ everywhere

67	Least Privilege	Services use admin roles	Per-service IAM roles + minimal permissions
68	Authentication Centralization	Each service has separate login	Central IAM/OAuth2/OpenID
69	Authorization Consistency	Auth rules differ per service	Central policy engine (OPA)
70	mTLS	Plain HTTP internally	Enforce encrypted service-to-service traffic
71	API Security	APIs open without controls	WAF + auth + quotas + validation
72	Input Validation	Trusting payloads	Validate schema and sanitize payloads
73	Dependency Governance	Random libs per service	Standardized dependency policy
74	Vulnerability Scanning	No container scan	SCA + image scanning in CI
75	Supply Chain Security	Unsigned images	Signing/attestation (cosign, SBOM)
76	Service Discovery	Hard-coded IPs	Service discovery via DNS/Consul/K8s
77	Resilient Discovery	Discovery failure breaks system	Cache discovery + retries
78	Load Balancing	Single endpoint	Client-side LB or service mesh LB
79	Contract Consistency	Inconsistent error codes	Standard error model + status mapping
80	API Standards	Different naming conventions	Company-wide API guidelines
81	Avoid Tight Coupling Libraries	Shared JAR runtime dependency	Prefer API contracts and independent libs
82	Version Pinning	Uncontrolled upgrades break services	Managed dependency versions
83	Polyglot Chaos	Too many languages	Standardize stack unless needed
84	Data Replication Rules	Copying DB tables	Share via events/read models

85	Caching Policy	Cache without invalidation	TTL + cache invalidation plan
86	Cache Aside	Cache used incorrectly	Use cache-aside + fallbacks
87	Session Management	Sticky sessions	Stateless auth tokens (JWT) or external session store
88	Stateless Services	In-memory user data	Store state in Redis/DB
89	Resource Limits	Unlimited CPU/memory	Enforce resource limits and quotas
90	Noisy Neighbor Control	One service hogs resources	Quotas, limits, priority classes
91	Multi-tenancy Isolation	Tenants share everything	Tenant isolation at data and compute
92	Regional Redundancy	One-region deployment	Multi-region DR + failover
93	Disaster Recovery	No DR tests	Regular DR drills + RTO/RPO defined
94	Chaos Engineering	No failure simulation	Inject outages and validate recovery
95	Auditability	No audit logs	Audit trail for critical actions
96	Compliance by Design	Compliance added later	Data retention, masking, access policies
97	Monitoring Dependencies	Only monitor your service	Monitor downstream dependencies too
98	Cost Optimization	Ignoring cost of microservices	Consolidate workloads + autoscale + right-size
99	Ownership & Runbooks	No documentation/support	Runbooks + on-call + escalation
100	Platform Engineering	Every team reinvents infra	Shared platform: templates, golden paths

If you want next, I can also provide:

✓ **Microservices Anti-pattern → Correct pattern mapping (short memorable names)**

Example: *Distributed Monolith* → *Event-driven architecture*

✓ **Microservices architecture diagram set (15 diagrams)**

✓ **Printable PDF cheat sheet (100-row table)**